```
> # ============================================================
  # TauTimp — Spherical Symmetry: Variable-Coefficient Wave
  Equation
  # Generalized Framework: Airy + d'Alembert Solutions
  # Author: Viorel Popescu
  # Maple Transcript v1.0
  # ============================================================

  restart;
  with(plots):
  with(plottools):
  Digits := 15:

  # ============================================================
  # SECTION 1: PHYSICAL PARAMETERS
  # ============================================================

  R  := 4.235518529e22:           # [m]  TauTimp boundary radius
  R2 := 2*R:                  # [m]  boundary for Case A
  c  := 3.0e8:                # [m/s] speed of light
  V1 := c:                    # Case 1: V = c
  V2 := 1.0e9 * c:            # Case 2: V = 10^9 * c (onduscular)
  V3 := 346.0:                # Case 3: V = 346 m/s
  (acoustic/phonon)

  printf("R  = %.6e m\n", R):
  printf("2R = %.6e m\n", R2):
  printf("V1 (light)      = %.4e m/s\n", V1):
  printf("V2 (onduscular) = %.4e m/s\n", V2):
  printf("V3 (acoustic)   = %.4e m/s\n", V3):

  # ============================================================
  # SECTION 2: AIRY FUNCTION ZEROS
  # ============================================================
  # Ai(-x) = 0  at x = a_n
  # Asymptotic: a_n ~ [3*Pi/8*(4n-1)]^(2/3)

  N_modes := 5:

  # Compute zeros of Ai(-x) = 0, i.e. Ai(x)=0 shifted
  # We use fsolve with asymptotic initial guesses
  airy_zero := proc(n::posint) local guess;
```

```
    guess := evalf((3*Pi/8*(4*n-1))^(2/3));
    fsolve(AiryAi(-x), x, x = guess - 0.3 .. guess + 0.3);
end proc:

a := Vector(N_modes):
for n from 1 to N_modes do
    a[n] := airy_zero(n);
    printf("a[%d] = %.8f\n", n, a[n]);
end do:

# ============================================================
# SECTION 3: EIGENVALUE QUANTIZATION
# ============================================================

# Wave numbers — Case A: boundary at 2R
k_A := proc(n) (2 * a[n]^(3/2)) / (3 * R2^(3/2)) * sqrt(R) end
proc:

# Wave numbers — Case B: boundary at R
k_B := proc(n) (2 * a[n]^(3/2)) / (3 * R^(3/2)) * sqrt(R) end
proc:

printf("\n--- Case A: boundary at 2R ---\n"):
for n from 1 to N_modes do
    printf("k_A[%d] = %.6e m^-1\n", n, evalf(k_A(n)));
end do:

printf("\n--- Case B: boundary at R ---\n"):
for n from 1 to N_modes do
    printf("k_B[%d] = %.6e m^-1\n", n, evalf(k_B(n)));
end do:

# Frequencies omega_n = k_n * V for all variants
printf("\n--- Frequencies omega = k*V ---\n"):
printf("%-6s %-22s %-22s %-22s %-22s %-22s %-22s\n",
    "Mode",
    "A,V=c", "A,V=10^9c", "A,V=346",
    "B,V=c", "B,V=10^9c", "B,V=346"):

for n from 1 to N_modes do
    printf("n=%d    %.4e    %.4e    %.4e    %.4e    %.4e
%.4e\n", n,
```

```
        evalf(k_A(n)*V1), evalf(k_A(n)*V2), evalf(k_A(n)*V3),
        evalf(k_B(n)*V1), evalf(k_B(n)*V2), evalf(k_B(n)*V3));
end do:

# ============================================================
# SECTION 4: RADIAL AIRY EIGENFUNCTION
# ============================================================

# Dimensionless Airy variable
zeta_A := proc(r_val, n) local kn;
    kn := k_A(n);
    evalf((3*kn/2)^(2/3) * r_val / R^(1/3));
end proc:

# Radial eigenfunction (normalized)
g_A := proc(r_val, n)
    evalf(AiryAi(-zeta_A(r_val, n)) / sqrt(r_val));
end proc:

# ============================================================
# SECTION 5: PLOTS — RADIAL EIGENFUNCTIONS
# ============================================================

Nr := 300:
r_min := 0.01 * R:
r_max := R2:
r_vals := [seq(r_min + (r_max - r_min)*(i-1)/(Nr-1), i = 1..Nr)]:

# Build data for first 3 modes
colors_modes := [red, blue, green]:
curve_list := []:
for n from 1 to 3 do
    data_n := [seq([r_vals[i]/R, g_A(r_vals[i], n)], i = 1..Nr)]:
    curve_list := [op(curve_list), listplot(data_n, color =
colors_modes[n],
        legend = cat("Mode n=", n), linestyle = solid)]:
end do:

p_radial := display(curve_list,
    title = "TauTimp: Radial Airy Eigenfunctions g_A(r,n) vs
r/R",
    labels = ["r/R", "g(r,n) [a.u.]"],
```

```
    labeldirections = [horizontal, vertical],
    axes = boxed,
    gridlines = true,
    size = [800, 400]):

print(p_radial):

# ============================================================
# SECTION 6: TIME EVOLUTION f(r,t) = g(r)*u(t)
# ============================================================
# Fix n=1 mode, show f(r,t) for Case A, V=c

omega_A1_c := k_A(1) * V1:
printf("\nomega_A1 (V=c) = %.6e rad/s\n", evalf(omega_A1_c)):

T_period := evalf(2*Pi / omega_A1_c):
printf("Period T = %.6e s\n", T_period):

# f(r,t) at t = 0, T/4, T/2
t_vals := [0, T_period/4, T_period/2]:
colors_t := [blue, purple, orange]:
labels_t := ["t=0", "t=T/4", "t=T/2"]:

curve_time := []:
for j from 1 to 3 do
    tj := t_vals[j];
    data_t := [seq([r_vals[i]/R,
        g_A(r_vals[i], 1) * cos(omega_A1_c * tj)],
        i = 1..Nr)]:
    curve_time := [op(curve_time), listplot(data_t,
        color = colors_t[j],
        legend = labels_t[j],
        linestyle = solid)]:
end do:

p_time_evo := display(curve_time,
    title = "f(r,t) = g_A(r,1)*cos(omega*t), Case A, V=c, n=1",
    labels = ["r/R", "f(r,t) [a.u.]"],
    axes = boxed, gridlines = true, size = [800, 400]):

print(p_time_evo):
```

```
# ============================================================
# SECTION 7: d'ALEMBERT SOLUTIONS — STANDING WAVES
# ============================================================
# f(r,t) = A * sin(n*Pi*(r - r0)/r0) * cos(omega_n * t)
# Case A: r0 = 2R, f(2R,0)=0
# Case B: r0 = R,  f(R, 0)=0

# Case A, d'Alembert, V=c
f_dA := proc(r_val, t_val, n_mode, V_val)
    local omega_n;
    omega_n := n_mode * Pi * V_val / R2;
    sin(n_mode * Pi * (r_val - R2) / R2) * cos(omega_n * t_val);
end proc:

# Case B, d'Alembert, V=c
f_dB := proc(r_val, t_val, n_mode, V_val)
    local omega_n;
    omega_n := n_mode * Pi * V_val / R;
    sin(n_mode * Pi * (r_val - R) / R) * cos(omega_n * t_val);
end proc:

# Plot: Case A d'Alembert at t=0, modes n=1,2,3
r_dA_min := R2 - 2*R:   # = 0
r_dA_max := R2:
r_dA_vals := [seq(0.001*R + (R2 - 0.001*R)*(i-1)/(Nr-1), i = 1..
Nr)]:

curve_dA := []:
for n from 1 to 3 do
    data_dA := [seq([r_dA_vals[i]/R,
        f_dA(r_dA_vals[i], 0, n, V1)], i = 1..Nr)]:
    curve_dA := [op(curve_dA), listplot(data_dA,
        color = colors_modes[n],
        legend = cat("n=", n),
        linestyle = solid)]:
end do:

p_dAlembert_A := display(curve_dA,
    title = "d'Alembert: f(r,0), Case A [f(2R,0)=0], V=c",
    labels = ["r/R", "f(r,0)"],
    axes = boxed, gridlines = true, size = [800, 400]):
```

```
print(p_dAlembert_A):

# Plot: Case B d'Alembert at t=0
r_dB_vals := [seq(R*(0.001 + (i-1)/(Nr-1)), i = 1..Nr)]:

curve_dB := []:
for n from 1 to 3 do
    data_dB := [seq([r_dB_vals[i]/R,
        f_dB(r_dB_vals[i], 0, n, V1)], i = 1..Nr)]:
    curve_dB := [op(curve_dB), listplot(data_dB,
        color = colors_modes[n],
        legend = cat("n=", n),
        linestyle = solid)]:
end do:

p_dAlembert_B := display(curve_dB,
    title = "d'Alembert: f(r,0), Case B [f(R,0)=0], V=c",
    labels = ["r/R", "f(r,0)"],
    axes = boxed, gridlines = true, size = [800, 400]):

print(p_dAlembert_B):

# ============================================================
# SECTION 8: VELOCITY COMPARISON PLOTS
# ============================================================
# Compare d'Alembert n=1 spatial profile for all 3 velocities
# (profile shape is identical — only frequencies differ)
# Show frequency spectrum instead

printf("\n--- d'Alembert Frequencies (Case A, n=1) ---\n"):
omega_dA_c  := Pi * V1 / R2:
omega_dA_o  := Pi * V2 / R2:
omega_dA_ac := Pi * V3 / R2:

printf("V=c       : omega = %.6e rad/s\n", evalf(omega_dA_c)):
printf("V=10^9*c  : omega = %.6e rad/s\n", evalf(omega_dA_o)):
printf("V=346 m/s : omega = %.6e rad/s\n", evalf(omega_dA_ac)):

printf("\n--- d'Alembert Frequencies (Case B, n=1) ---\n"):
omega_dB_c  := Pi * V1 / R:
omega_dB_o  := Pi * V2 / R:
omega_dB_ac := Pi * V3 / R:
```

```
printf("V=c       : omega = %.6e rad/s\n", evalf(omega_dB_c)):
printf("V=10^9*c  : omega = %.6e rad/s\n", evalf(omega_dB_o)):
printf("V=346 m/s : omega = %.6e rad/s\n", evalf(omega_dB_ac)):

# Time evolution at fixed r = R (midpoint) for Case A, n=1, all V
# Rescale time by T_c = 2*Pi/omega_c for each velocity

printf("\nPlotting time evolution at r=R, Case A, n=1 ...\n"):

t_plot_pts := 500:
r_fixed := R:

# For V=c: plot over 3 periods
T_c  := evalf(2*Pi / omega_dA_c):
t_c  := [seq(T_c * (i-1) / t_plot_pts, i = 1..t_plot_pts+1)]:
d_c  := [seq([t_c[i]/T_c,
    f_dA(r_fixed, t_c[i], 1, V1)], i = 1..t_plot_pts+1)]:

# For acoustic: much shorter period
T_ac := evalf(2*Pi / omega_dA_ac):
t_ac := [seq(T_ac * 3 * (i-1) / t_plot_pts, i = 1..t_plot_pts+1)]
:
d_ac := [seq([t_ac[i]/T_ac,
    f_dA(r_fixed, t_ac[i], 1, V3)], i = 1..t_plot_pts+1)]:

p_time_c := listplot(d_c, color = blue,
    title = "f(R, t): Case A, V=c, n=1  [t in units of T_c]",
    labels = ["t/T", "f(R,t)"],
    axes = boxed, gridlines = true, size = [800, 300]):

p_time_ac := listplot(d_ac, color = green,
    title = "f(R, t): Case A, V=346 m/s, n=1  [t in units of
T_acoustic]",
    labels = ["t/T", "f(R,t)"],
    axes = boxed, gridlines = true, size = [800, 300]):

print(p_time_c):
print(p_time_ac):

# ============================================================
# SECTION 9: FLUX DENSITY phi(r,t) = f(r,t)/(4*Pi*r^2)
# ============================================================
```

```
phi_A := proc(r_val, t_val, n_mode, V_val)
    local fn;
    fn := g_A(r_val, n_mode) * cos(k_A(n_mode) * V_val * t_val);
    fn / (4 * Pi * r_val^2);
end proc:

# Plot flux at t=0, n=1, Airy mode, Case A
r_flux_vals := [seq(0.01*R + (R2 - 0.01*R)*(i-1)/(Nr-1), i = 1..
Nr)]:
data_flux := [seq([r_flux_vals[i]/R,
    evalf(phi_A(r_flux_vals[i], 0, 1, V1))], i = 1..Nr)]:

p_flux := listplot(data_flux, color = darkred,
    title = "Flux Phi(r,0) = f(r,0)/(4*Pi*r^2), Airy n=1, Case
A",
    labels = ["r/R", "Phi [a.u.]"],
    axes = boxed, gridlines = true, size = [800, 400]):

print(p_flux):

# ============================================================
# SECTION 10: COMBINED SUMMARY TABLE
# ============================================================

printf("\n=== GENERALIZED FRAMEWORK SUMMARY ===\n"):
printf("R = %.4e m\n\n", R):
printf("%-5s %-10s %-10s %-12s %-22s %-22s\n",
    "Case", "BC", "V [m/s]", "Method", "omega_1 [rad/s]", "T_1
[s]"):
printf("%s\n", StringTools:-Repeat("-", 88)):

cases := [
    ["A", "f(2R,0)=0", "c", "Airy",        k_A(1)*V1, 2*Pi/(k_A
(1)*V1)],
    ["A", "f(2R,0)=0", "10^9*c", "Airy",   k_A(1)*V2, 2*Pi/(k_A
(1)*V2)],
    ["A", "f(2R,0)=0", "346", "Airy",       k_A(1)*V3, 2*Pi/(k_A
(1)*V3)],
    ["A", "f(2R,0)=0", "c", "d'Alembert",   Pi*V1/R2,  2*R2/V1],
    ["A", "f(2R,0)=0", "10^9*c","d'Alembert",Pi*V2/R2, 2*R2/V2],
    ["A", "f(2R,0)=0", "346","d'Alembert",  Pi*V3/R2,  2*R2/V3],
    ["B", "f(R,0)=0",  "c", "Airy",         k_B(1)*V1, 2*Pi/(k_B
```

```
(1)*V1)],
    ["B", "f(R,0)=0",  "10^9*c", "Airy",    k_B(1)*V2, 2*Pi/(k_B
(1)*V2)],
    ["B", "f(R,0)=0",  "346", "Airy",       k_B(1)*V3, 2*Pi/(k_B
(1)*V3)],
    ["B", "f(R,0)=0",  "c", "d'Alembert",   Pi*V1/R,   2*R/V1],
    ["B", "f(R,0)=0",  "10^9*c","d'Alembert",Pi*V2/R,  2*R/V2],
    ["B", "f(R,0)=0",  "346","d'Alembert",  Pi*V3/R,   2*R/V3]
]:

for row in cases do
    printf("%-5s %-10s %-10s %-12s %.6e          %.6e\n",
        row[1], row[2], row[3], row[4], evalf(row[5]), evalf(row
[6]));
end do:

# ============================================================
# SECTION 11: DENSITY PLOT — f(r,t) 2D MAP
# ============================================================
# Airy n=1, Case A, V=c: plot f as heatmap over (r/R, t/T)

T1 := evalf(2*Pi / (k_A(1)*V1)):
Nr2 := 80:
Nt2 := 80:

r_grid := [seq(0.01*R + (R2-0.01*R)*(i-1)/(Nr2-1), i=1..Nr2)]:
t_grid := [seq(T1*(j-1)/(Nt2-1), j=1..Nt2)]:

M_f := Matrix(Nt2, Nr2):
for j from 1 to Nt2 do
    for i from 1 to Nr2 do
        M_f[j,i] := evalf(g_A(r_grid[i], 1) * cos(k_A(1)*V1*
t_grid[j]));
    end do:
end do:

# Convert matrix to listplot3d data
data3d := []:
for j from 1 to Nt2 do
    for i from 1 to Nr2 do
        data3d := [op(data3d),
            [r_grid[i]/R, t_grid[j]/T1, M_f[j,i]]]:
```

```
    end do:
end do:

p_3d := pointplot3d(data3d,
    symbol = point, symbolsize = 3,
    color = z,
    title = "f(r,t): Airy n=1, Case A, V=c  [axes: r/R, t/T,
amplitude]",
    labels = ["r/R", "t/T", "f"],
    axes = boxed,
    orientation = [45, 70],
    size = [800, 500]):

print(p_3d):

# ============================================================
# SECTION 12: ASYMPTOTIC WKB COMPARISON
# ============================================================

# WKB approximation of Ai(-zeta) for large zeta
Ai_WKB := proc(zeta_val)
    (1/(sqrt(Pi)*zeta_val^(1/4))) * sin(2/3 * zeta_val^(3/2) +
Pi/4);
end proc:

# Compare exact Ai vs WKB at n=1
r_wkb := [seq(0.05*R + (R2 - 0.05*R)*(i-1)/(Nr-1), i = 1..Nr)]:

data_exact := [seq([r_wkb[i]/R,
    evalf(AiryAi(-zeta_A(r_wkb[i], 1)) / sqrt(r_wkb[i]))], i = 1.
.Nr)]:
data_wkb   := [seq([r_wkb[i]/R,
    Ai_WKB(zeta_A(r_wkb[i], 1)) / sqrt(r_wkb[i])], i = 1..Nr)]:

p_wkb := display(
    listplot(data_exact, color = blue,  legend = "Exact Airy"),
    listplot(data_wkb,   color = red,   legend = "WKB", linestyle
= dash),
    title = "Airy Exact vs WKB: g(r,n=1), Case A",
    labels = ["r/R", "g(r)"],
    axes = boxed, gridlines = true, size = [800, 400]):
```

```
print(p_wkb):

# ============================================================
# SECTION 13: ANGULAR RECONSTRUCTION H(r,theta,t)
# ============================================================
# H = sum_l f_l(r,t) * P_l(cos(theta))
# Use l=0,1,2 with equal amplitude weights D_l = 1

H_field := proc(r_val, theta_val, t_val, V_val)
    local H_sum, l, P_l, f_l_rt;
    H_sum := 0;
    for l from 0 to 2 do
        # Use Airy n=1 mode for each l with weight 1
        f_l_rt := g_A(r_val, 1) * cos(k_A(1) * V_val * t_val);
        # Legendre polynomial P_l(cos(theta))
        P_l := LegendreP(l, cos(theta_val));
        H_sum := H_sum + f_l_rt * P_l;
    end do;
    evalf(H_sum);
end proc:

# Plot H(r,theta,0) as surface in (r, theta)
Nr_H := 50: Nth := 50:
r_H := [seq(0.01*R + (R2-0.01*R)*(i-1)/(Nr_H-1), i=1..Nr_H)]:
th_H := [seq(Pi*(j-1)/(Nth-1), j=1..Nth)]:

data_H := []:
for i from 1 to Nr_H do
    for j from 1 to Nth do
        data_H := [op(data_H),
            [r_H[i]/R, th_H[j],
             H_field(r_H[i], th_H[j], 0, V1)]]:
    end do:
end do:

p_H := pointplot3d(data_H,
    symbol = point, symbolsize = 2,
    color = z,
    title = "H(r,theta,0): l=0,1,2 modes, V=c",
    labels = ["r/R", "theta", "H"],
    axes = boxed, orientation = [30, 60],
    size = [800, 500]):
```

```
print(p_H):

# ============================================================
# DONE
# ============================================================

printf("\n=== TauTimp Maple Transcript Complete ===\n"):
printf("Files generated: TauTimp_SphericalSymmetry.md +
TauTimp_SphericalSymmetry.mpl\n"):
```

```
R  = 4.235519e+22 m
2R = 8.471037e+22 m
V1 (light)      = 3.0000e+08 m/s
V2 (onduscular) = 3.0000e+17 m/s
V3 (acoustic)   = 3.4600e+02 m/s
a[1] = 2.33810741
a[2] = 4.08794944
a[3] = 5.52055983
a[4] = 6.78670809
a[5] = 7.94413359

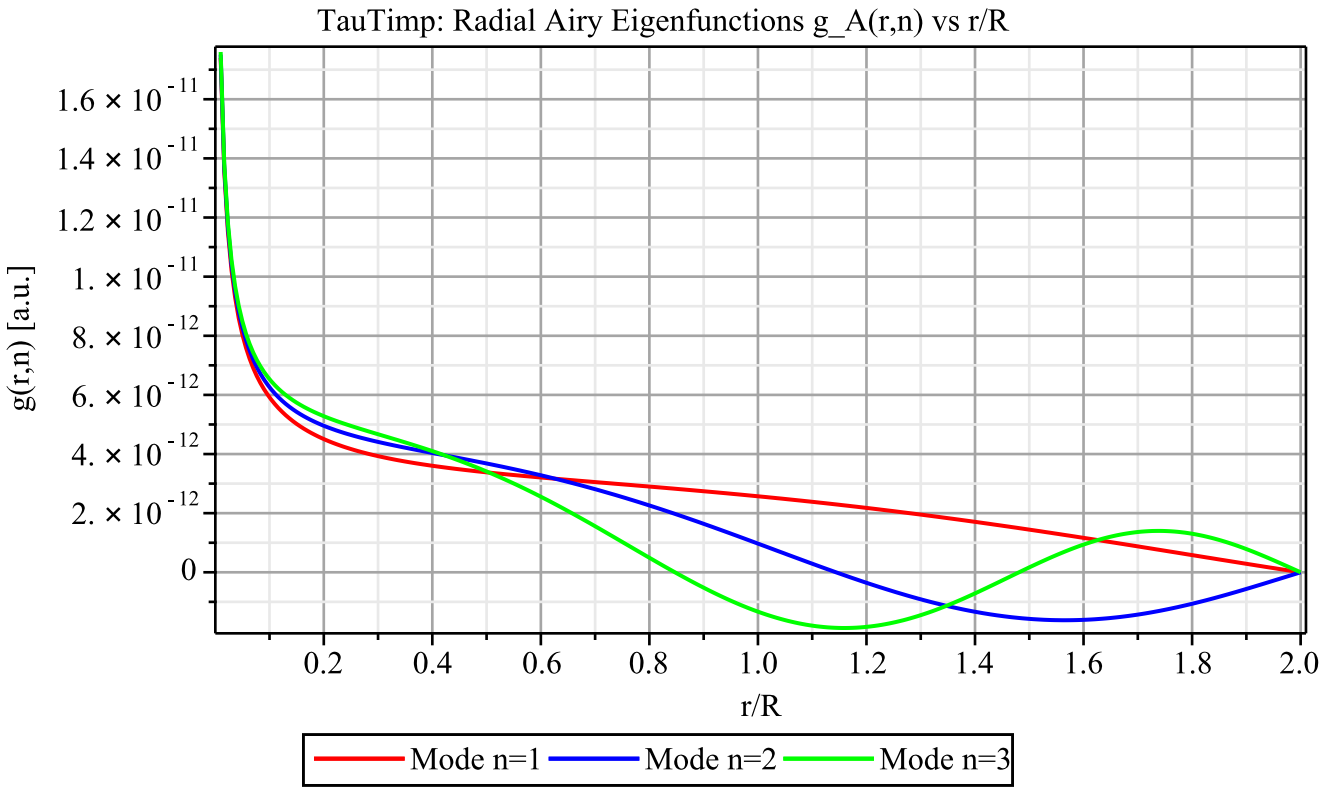
--- Case A: boundary at 2R ---
k_A[1] = 1.989545e-23 m^-1
k_A[2] = 4.599551e-23 m^-1
k_A[3] = 7.218249e-23 m^-1
k_A[4] = 9.838886e-23 m^-1
k_A[5] = 1.246026e-22 m^-1

--- Case B: boundary at R ---
k_B[1] = 5.627284e-23 m^-1
k_B[2] = 1.300949e-22 m^-1
k_B[3] = 2.041629e-22 m^-1
k_B[4] = 2.782857e-22 m^-1
k_B[5] = 3.524293e-22 m^-1

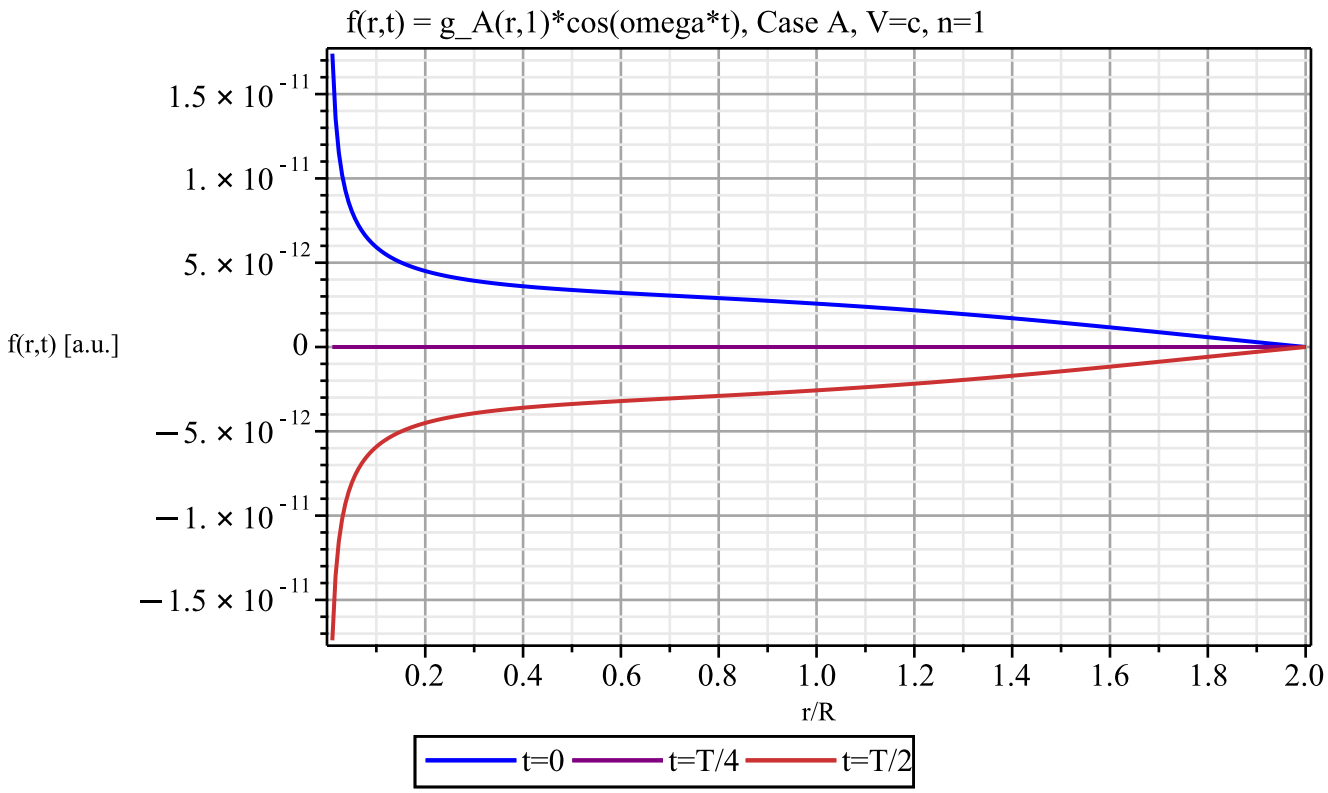
--- Frequencies omega = k*V ---
Mode   A,V=c                  A,V=10^9c              A,V=346
      B,V=c                  B,V=10^9c              B,V=346

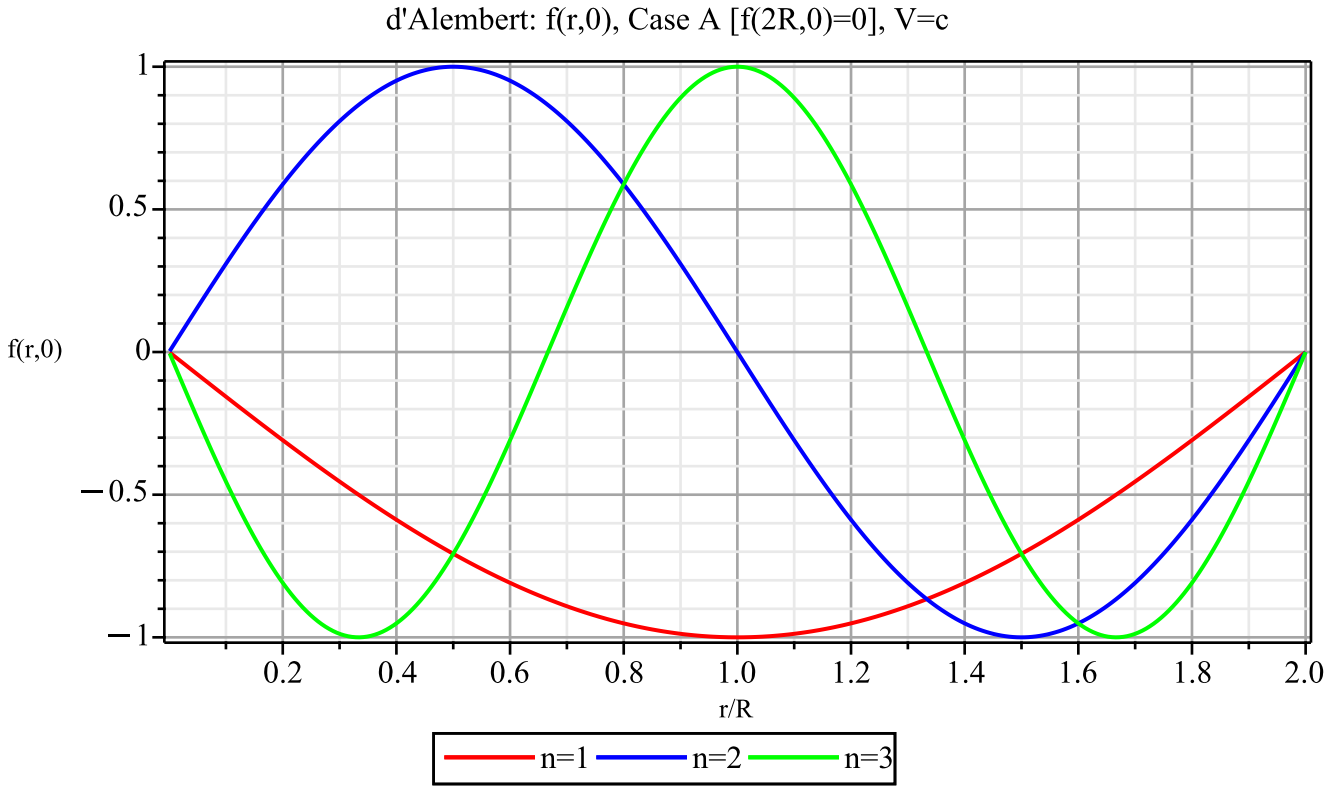
n=1    5.9686e-15    5.9686e-06    6.8838e-21    1.6882e-14
1.6882e-05    1.9470e-20
n=2    1.3799e-14    1.3799e-05    1.5914e-20    3.9028e-14
3.9028e-05    4.5013e-20
```

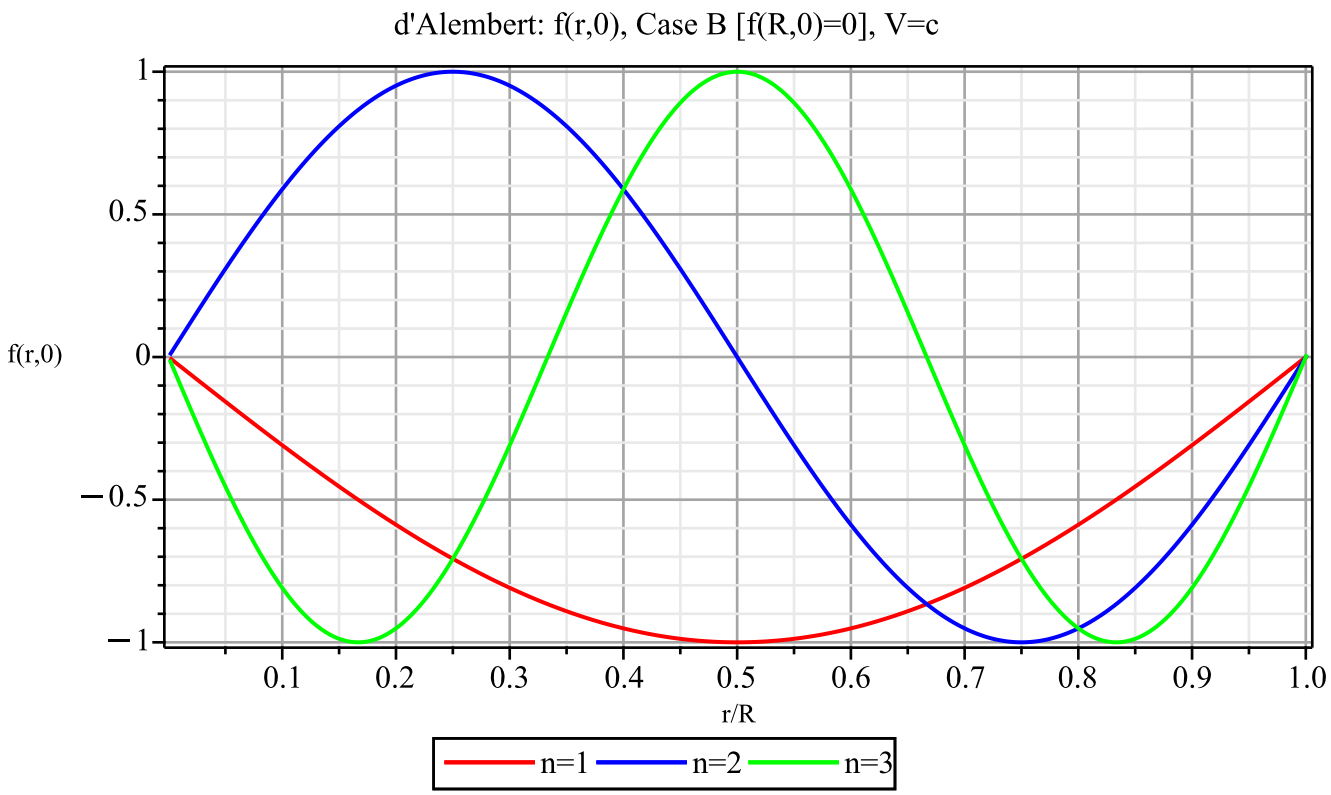
```
n=3    2.1655e-14    2.1655e-05    2.4975e-20    6.1249e-14
6.1249e-05    7.0640e-20
n=4    2.9517e-14    2.9517e-05    3.4043e-20    8.3486e-14
8.3486e-05    9.6287e-20
n=5    3.7381e-14    3.7381e-05    4.3112e-20    1.0573e-13
1.0573e-04    1.2194e-19
```




```
omega_A1 (V=c) = 5.968636e-15 rad/s
Period T = 1.052700e+15 s
```


f(r,t) = g_A(r,1)*cos(omega*t), Case A, V=c, n=1
f(r,t) [a.u.]
1.5 × 10 -11
1. × 10 -11
5. × 10 -12
0
−5. × 10 -12
−1. × 10 -11
−1.5 × 10 -11
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
2.0
r/R
t=0
t=T/4
t=T/2

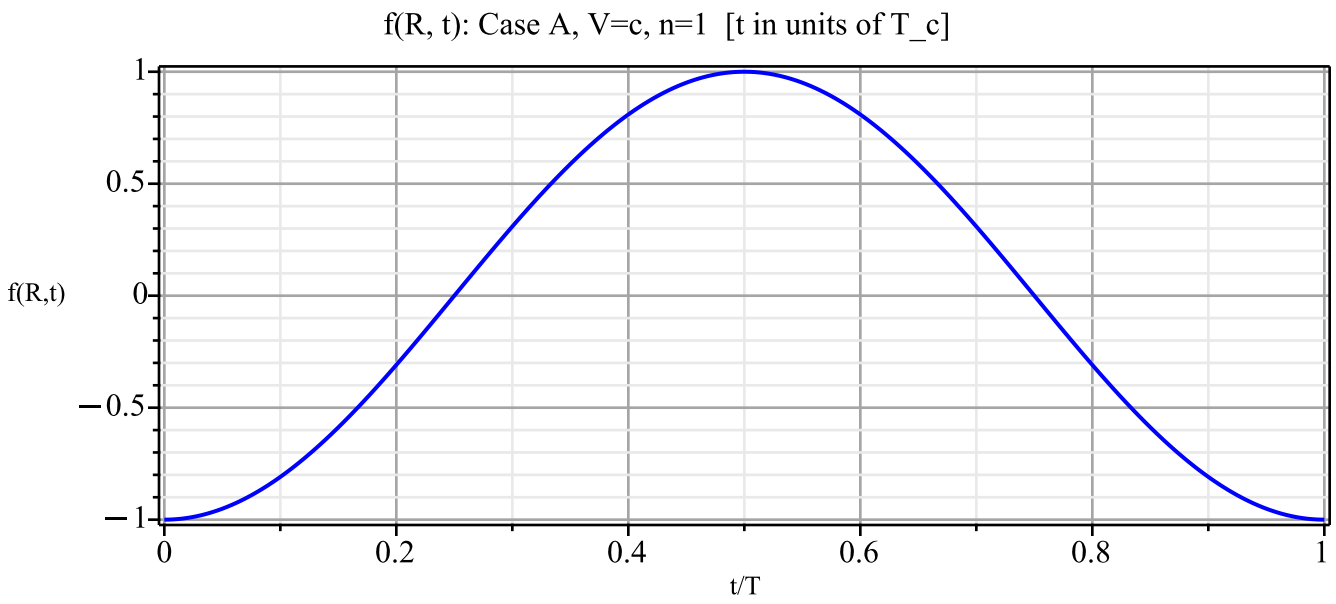

d'Alembert: f(r,0), Case A [f(2R,0)=0], V=c
f(r,0)
1
0.5
0
−0.5
−1
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
2.0
r/R
n=1
n=2
n=3

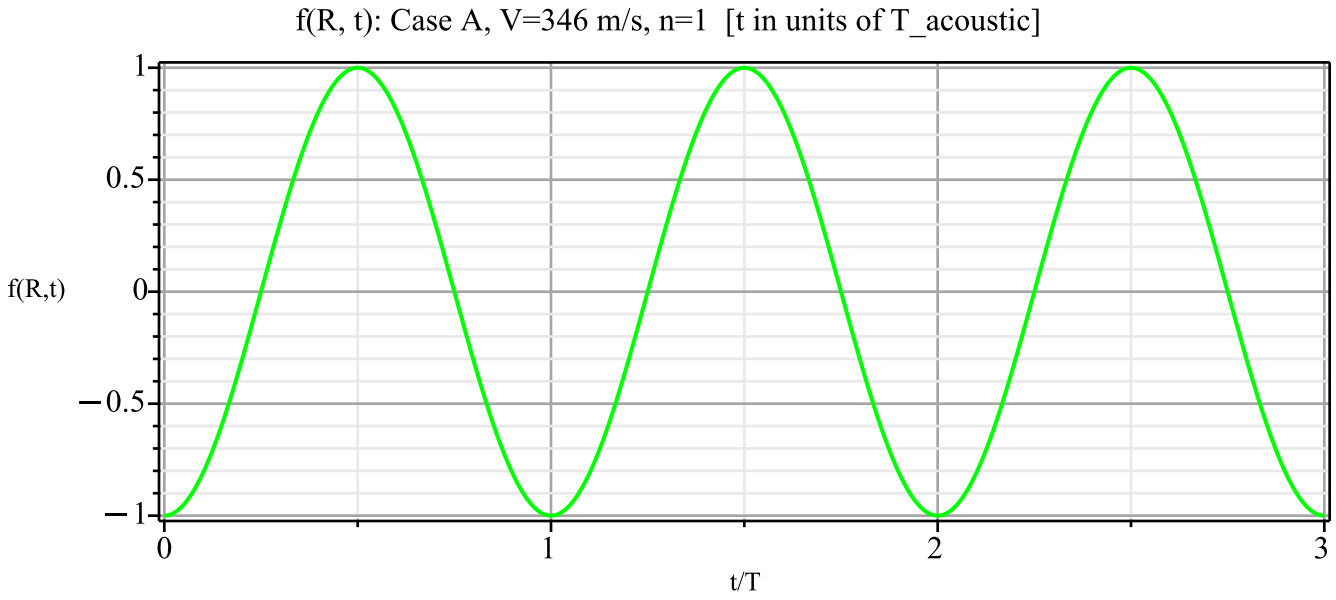


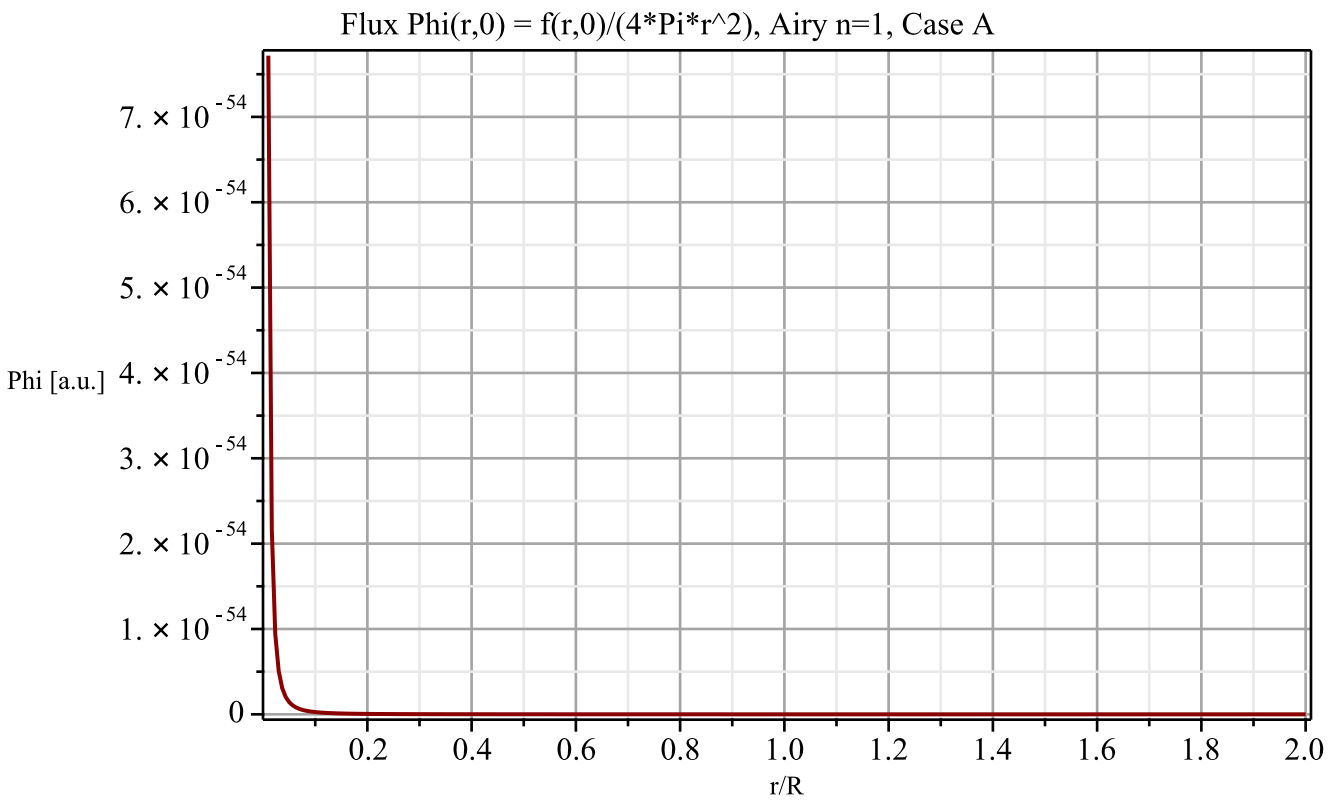

```
--- d'Alembert Frequencies (Case A, n=1) ---
V=c        : omega = 1.112588e-14 rad/s
V=10^9*c   : omega = 1.112588e-05 rad/s
V=346 m/s  : omega = 1.283185e-20 rad/s

--- d'Alembert Frequencies (Case B, n=1) ---
V=c        : omega = 2.225177e-14 rad/s
V=10^9*c   : omega = 2.225177e-05 rad/s
V=346 m/s  : omega = 2.566371e-20 rad/s

Plotting time evolution at r=R, Case A, n=1 ...
```


f(R, t): Case A, V=c, n=1 [t in units of T_c]
f(R,t)
1
0.5
0
−0.5
−1
0
0.2
0.4
0.6
0.8
1
t/T


f(R, t): Case A, V=346 m/s, n=1 [t in units of T_acoustic]
f(R,t)
1
0.5
0
−0.5
−1
0
1
2
3
t/T



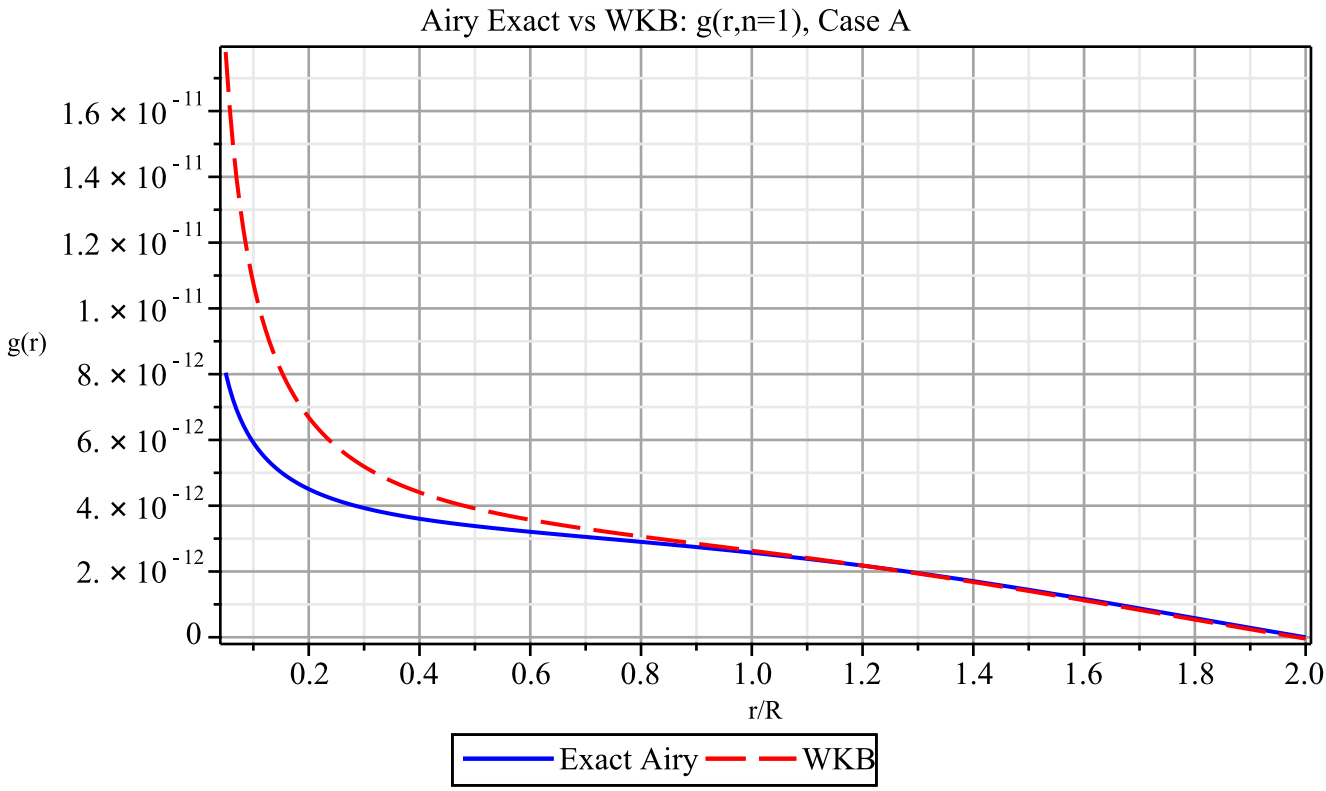

```
=== GENERALIZED FRAMEWORK SUMMARY ===
R = 4.2355e+22 m

Case  BC         V [m/s]    Method       omega_1 [rad/s]        T_1
[s]
-------------------------------------------------------------------
------------------
A     f(2R,0)=0  c          Airy         5.968636e-15
1.052700e+15
A     f(2R,0)=0  10^9*c     Airy         5.968636e-06
1.052700e+06
A     f(2R,0)=0  346        Airy         6.883827e-21
9.127460e+20
A     f(2R,0)=0  c          d'Alembert   1.112588e-14
5.647358e+14
A     f(2R,0)=0  10^9*c     d'Alembert   1.112588e-05
5.647358e+05
A     f(2R,0)=0  346        d'Alembert   1.283185e-20
4.896553e+20
B     f(R,0)=0   c          Airy         1.688185e-14
3.721858e+14
B     f(R,0)=0   10^9*c     Airy         1.688185e-05
```

```
3.721858e+05
B     f(R,0)=0    346         Airy          1.947040e-20
3.227044e+20
B     f(R,0)=0    c           d'Alembert    2.225177e-14
2.823679e+14
B     f(R,0)=0    10^9*c      d'Alembert    2.225177e-05
2.823679e+05
B     f(R,0)=0    346         d'Alembert    2.566371e-20
2.448277e+20
```

Error, (in plots:-pointplot3d) invalid color specification: z

p_3d



Error, (in plots:-pointplot3d) invalid color specification: z

p_H

```
=== TauTimp Maple Transcript Complete ===
Files generated: TauTimp_SphericalSymmetry.md +
TauTimp_SphericalSymmetry.mpl
```